05 · Tools

Siddharth M. Bhatia · siddharth@sidmb.com
July 1, 2026

Contents

Define and register a tool 1
Give a participant tools 2
  Live participant 2
  On a participant 2
What the loop does 2

A `Tool` is a capability a participant can invoke mid-turn. The harness runs a uniform tool loop; each family parses its own native call format (Hermes/Qwen `<tool_call>` JSON, Gemma ```` ```tool_code ````, Llama `◁python_tag▷`). Tools hold live callables, so they are **not** serializable — templates store tool *names* and resolve them against a `ToolRegistry` at build time (mirroring how models resolve from ids).

Define and register a tool

```
from interlens import Tool, DEFAULT_REGISTRY

class Calculator(Tool):
    name = "calculator"

    @property
    def schema(self) -> dict:
        return {
            "type": "function",
            "function": {
                "name": "calculator",
                "description": "Evaluate a basic arithmetic expression and
     return the result.",
                "parameters": {
                    "type": "object",
                    "properties": {"expression": {"type": "string",
     "description": "e.g. '2 * (3 + 4)'"}},
                    "required": ["expression"],
                },
            },
        }
```

```
    def __call__(self, expression: str) -> str:
        return str(eval(expression, {"__builtins__": {}}))   # sandbox
     eval for the example only
DEFAULT_REGISTRY.register(Calculator())   # now resolvable by the name
     "calculator"
```

The `schema` is passed straight to `apply_chat_template(tools=...)`, so use the standard function-calling shape.

Give a participant tools

Live participant

```
from interlens import AutoModelParticipant
solver = AutoModelParticipant.from_pretrained(
    "Qwen/Qwen2.5-7B-Instruct", name="solver",
    tools=(Calculator(),),     # the tool objects
    max_tool_iters=4,          # bound the call→result→call loop
)
```

On a participant

```
from interlens import Conversation, AutoModelParticipant
solver = AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-7B-Instruct",
     name="solver",
                                          tools=(Calculator(),),
     max_tool_iters=4)
conv = Conversation(participants=[solver], shared_context="Compute 17 * 23
     using your tool, then explain.")
conv.run(turns=2)
```

Tools are resolved by name from `DEFAULT_REGISTRY` only when a saved conversation is reloaded (`Conversation.load`) — `save` stores each participant's tools by name; pass `registry=` there for a scoped registry.

What the loop does

For each turn the model may emit tool calls; the harness executes each, appends the call + result to a *private* working view, and lets the model react — up to `max_tool_iters` times. Only the **final natural-language**

message reaches the shared transcript; the full call/result trail is kept in `msg.metadata["tool_trail"]`. Tool exceptions are returned to the model as `error: …` results (data, not crashes), so a failing tool doesn't kill the run.

```
msg = conv.transcript[-1]
for step in msg.metadata.get("tool_trail", []):
    print(step)   # {"name": "calculator", "arguments": {...}, "output":
     "391", "error": False}
```

Use a scoped registry instead of the global one by passing `registry=` to `Conversation.load`.

Next: hooks.

© 2026 Siddharth M. Bhatia. All rights reserved. Interlens is licensed under the GNU Affero General Public License v3.0.